

INGENIERÍA Y CALIDAD DEL SOFTWARE

TRABAJO PRÁCTICO N° 3

Gestión de Configuración y Pipeline CI/CD 2026

Objetivo

Establecer una estrategia de gestión de la configuración mediante un modelo de ramificación profesional en Git, contenerizar la API .NET del e-commerce utilizando Docker, y automatizar el ciclo de Integración Continua y Despliegue Continuo (CI/CD) mediante GitHub Actions hacia una plataforma de hosting gratuita. Todo el trabajo técnico debe ser planificado, estimado y gestionado de manera colaborativa en el tablero ágil del equipo (Sprint 2) bajo las políticas de calidad definidas (DoR y DoD)

Enunciado

A partir del sistema e-commerce en desarrollo (.NET API + React + Base de Datos), los equipos deberán modernizar la gestión del código y la infraestructura de entregas a lo largo de las 3 clases prácticas asignadas:

- **1: Modelo de Ramificación en Git y Políticas de Repositorio**
 - **Ramas Principales:** Configurar en GitHub las ramas `main` (producción/estable) y `development` (para integración continua).
 - **Estrategia de Branching:** Adoptar un flujo de trabajo basado en ramas por característica (feature/nombre-funcionalidad) o corrección (hotfix/descripción). No se permite realizar *commits* directos sobre `main` ni `development`.
 - **Protección de Ramas (Branch Protection Rules):** Configurar reglas en GitHub que exijan obligatoriamente la aprobación de al menos una revisión de código por un par (*Peer Review*) mediante *Pull Request* (PR) antes de fusionar cualquier cambio en `development` o `main`.
- **2: Dockerización de la API de .NET**
 - **Dockerfile Multi-Stage:** Crear un archivo `Dockerfile` optimizado para la API de .NET en el repositorio, utilizando etapas diferenciadas de compilación/publicación (*build*) y de ejecución liviana (*runtime*).
 - **Inyección de Variables de Entorno:** Configurar la aplicación para URL y `environment`.
 - **Verificación Local:** Probar el correcto funcionamiento del contenedor de forma local para garantizar que la API inicie y responda a las peticiones correctamente.
- **3: Configuración de GitHub Actions y Despliegue en Hosting Gratuito**
 - **Workflow de CI/CD (GitHub Actions):** Diseñar e implementar el archivo de automatización `.github/workflows/deploy.yml` que se ejecute ante eventos de *push* o *merge* en las ramas de integración/despliegue.
 - **Etapas del Pipeline:** Restauración de paquetes, compilación limpia, construcción de la imagen Docker (o empaquetado del artefacto) y despliegue automático.
 - **Despliegue a Hosting Gratuito (Estrategia Costo Cero):** Publicar la API en un servidor accesible públicamente utilizando una de las opciones sugeridas por la

cátedra:

- Azure App Service (Plan Free).
- SQL Database (Plan Free).
- **4: Habilitar Swagger**
 - Se debe habilitar Swagger para que funcione desde la URL (domain) que ofrece Azure.

Entregables Requeridos

- Creación de tarjetas en Trello y estimaciones, preparar Sprint 2.
- Stories en Done, y respaldado por acceso al repositorio.
- Acceso a la API, se debe habilitar Swagger.
- Se debe documentar el proceso completo, indicando estrategia de ramificaciones, dockerfile, pipeline, conexión entre API y base de datos.